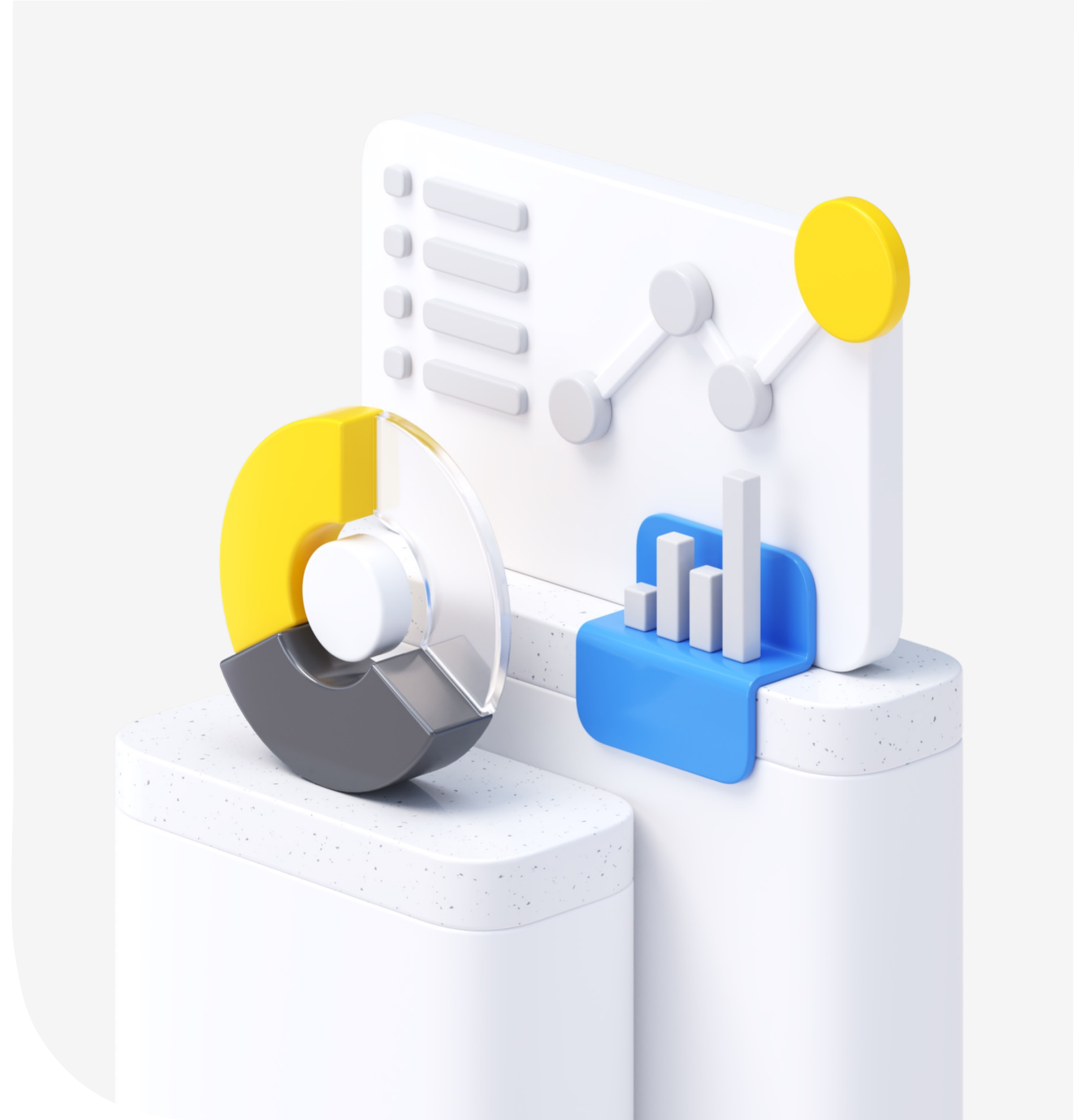




Лекция 6

Subscripts, Generics, Code Style



План

Subscript

- Что такое обращение по индексу?
- Зачем?

Generics

- Что такое обобщенные типы?
- Где и когда использовать?
- Инструменты

Code Style

- Зачем нужен?
- Практики и принципы

Subscript

Что это такое?

```
let dictionary: [String: Int] = ["Вася": 22]
```

```
let array: [String] = ["Петя"]
```

```
dictionary["Вася"] // возвращает Int?
```

```
array[0] // Возвращает String, упадет если промахнулись
```

- Позволяют обращаться к элементам коллекций, списков и других структур данных, не создавая дополнительный метод доступа.
- Они обеспечивают более удобный и лаконичный синтаксис.

Subscript

Синтаксис

```
subscript(index: Int) -> ElementType {  
  
    get { /*...*/ }  
  
    set(newValue) { /*...*/ }  
  
}
```

- Может быть добавлен только внутри объявления типа или через **extension**
- Могут быть как read/write так и read-only
- Имя переменной в **set** = newValue если не указано иное

Subscript

Пример

```
extension Array {  
    subscript(safe index: Int) -> Element? {  
        guard indices.contains(index) else {  
            return nil  
        }  
        return self[index]  
    }  
}  
  
let array: [String] = ["Петя"]  
  
print(array[safe: 3]) // nil
```

Subscript

Пример

```
class Matrix {  
  
    let rows: Int, columns: Int  
  
    var grid: [Double]  
  
    init(rows: Int, columns: Int) {  
  
        self.rows = rows  
  
        self.columns = columns  
  
        grid = Array(repeating: 0.0, count: rows * columns)  
  
    }  
  
    subscript(row: Int, column: Int) -> Double {  
  
        get { return grid[(row * columns) + column] }  
  
        set { grid[(row * columns) + column] = newValue }  
  
    }  
  
}  
  
let matrix = Matrix(rows: 2, columns: 2)  
  
matrix[0, 1] = 5.0  
  
print(matrix[0, 1]) // Вывод: 5.0
```

Subscript

Вывод

- Упрощение доступа к элементам коллекций
- Читаемость кода
- Возможность создания более интуитивных интерфейсов

Generics – Обобщенные типы

Generics

Что такое?

Generic типы позволяют создавать универсальные функции и типы.

- Могут работать с любыми типами данных.
- Могут накладывать ограничение на типы данных
- Обеспечивают гибкость и переиспользуемость кода
- Работают в Compile-time

Generics

Что-то знакомое

```
struct Array<Element>
```

```
enum Optional<Wrapped>
```

```
struct Dictionary<Key, Value> where Key : Hashable
```

```
class Foo<Bar>
```

Generics

Функции

```
func swap<T>(_ a: inout T, _ b: inout T) {  
    let temp = a  
  
    a = b  
  
    b = temp  
  
}  
  
var a = 5  
  
var b = 10  
  
swap(&a, &b)  
  
// Теперь a == 10, b == 5
```

Generics

Ограничения

```
extension Array where Element: AdditiveArithmetic {  
  
    func sum() -> Element {  
  
        reduce(.zero, +)  
  
    }  
  
}
```

Или

```
func sum<T: AdditiveArithmetic>(_ array: [T]) -> T {  
  
    array.reduce(.zero, +)  
  
}
```

Или

```
func sum<T>(_ array: [T]) -> T where T: AdditiveArithmetic {  
  
    array.reduce(.zero, +)  
  
}
```


Generics

Задача

```
protocol A {  
    var name: String { get }  
}  
  
func genericCompare<T: A>(_ val1: T, _ val2: T) -> Bool {  
    return val1.name == val2.name  
}  
  
func plainCompare(_ val1: A, _ val2: A) -> Bool {  
    return val1.name == val2.name  
}
```

Существуют ли различия между genericCompare и plainCompare? В чем выражаются?

Generics

Задача

```
protocol A {  
    var name: String { get }  
}  
  
func genericCompare<T: A>(_ val1: T, _ val2: T) -> Bool {  
    return val1.name == val2.name  
}  
  
func plainCompare(_ val1: A, _ val2: A) -> Bool {  
    return val1.name == val2.name  
}
```

```
struct Test1: A {  
    let name = "test"  
}
```

```
struct Test2: A {  
    let name = "test"  
}
```

```
let test1 = Test1()
```

```
let test2 = Test2()
```

```
print(plainCompare(test1, test2))
```

```
print(genericCompare(test1, test2))
```

Generics

Задача

```
protocol A {  
    var name: String { get }  
}  
  
func genericCompare<T: A>(_ val1: T, _ val2: T) -> Bool {  
    return val1.name == val2.name  
}  
  
func plainCompare(_ val1: A, _ val2: A) -> Bool {  
    return val1.name == val2.name  
}  
  
struct Test1: A {  
    let name = "test"  
}  
  
struct Test2: A {  
    let name = "test"  
}  
  
let test1 = Test1()  
let test2 = Test2()  
  
print(plainCompare(test1, test2))  
  
print(genericCompare(test1, test2))  
  
// error: conflicting arguments to generic parameter 'T' ('Test1' vs.  
// 'Test2')
```

Generics

Задача

```
protocol A {  
    var name: String { get }  
}  
  
//func genericCompare<T: A>(_ val1: T, _ val2: T) -> Bool {  
  
//    return val1.name == val2.name  
  
//}  
  
func genericCompare<T: A, U: A>(_ val1: T, _ val2: U) -> Bool {  
    val1.name == val2.name  
}  
  
func plainCompare(_ val1: A, _ val2: A) -> Bool {  
    val1.name == val2.name  
}
```


Generics

Выводы

- Позволяют создавать гибкие и переиспользуемые компоненты
- Ограничения помогают обеспечить безопасность типов и избежать ошибок

Вопросы?

Code style

Что и зачем?

Стиль кода – набор договоренностей о формате кода.

- Улучшает читаемость кода
- Упрощает поддержку
- Минимизирует количество ошибок и время на их исправление

KISS

Keep it simple stupid

- Ваши методы должны быть небольшими (не больше 40-50 строк).
- Каждый метод решает одну проблему.
- При модификации кода в будущем не должно возникнуть трудностей.
- Система работает лучше всего, если она не усложняется без надобности.
- Не устанавливайте целую библиотеку ради одной функции из неё.

DRY

Don't repeat yourself

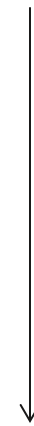
- Избегайте копирования кода.
- Выносите общую логику.
- Прежде чем добавлять функционал, проверьте в проекте, может, он уже создан.

DRY

Don't repeat yourself

```
func printUserName(user: User) {  
    print("User name: \(user.name)")  
}
```

```
func printAdminName(admin: Admin) {  
    print("Admin name: \(admin.name)")  
}
```



```
func printName(of person: Named) {  
    print("Name: \(person.name)")  
}
```


YAGNI

You aren't gonna need it

- Реализуйте только то, что нужно здесь и сейчас
- Подчищайте ненужный код
- Не добавляйте новый функционал, о котором его не просят

Литература

- [Пример код-стайла](#)
- [Дженерики \(часть 1\)](#)
- [Дженерики \(часть 2\)](#)
- [Сабскрипты от Apple \(eng\)](#)



Спасибо!

